

CPPDescent: A C++ library for the creation of K-NN graphs from multi-dimensional datasets

Software Development for Computing Systems, Winter 2023-2024

Department of Informatics and Telecommunications, University of Athens

Konstantinos Chousos*

Anastasios-Phaedon Seitanidis†

January 18, 2024

Abstract

This library is a C++ implementation of the “NN-Descent” algorithm by Dong et al. [4], with a few improvements and optimisations (e.g. usage of random projection trees [3]). This implementation serves as an entry for the [ACM SIGMOD 2023 competition](#). It was developed as part of the *Software Development for Computing Systems* course of the Department of Informatics and Telecommunications, taught in the winter of 2023 [9]. The source code is available at <https://github.com/kchousos/CPPDescent> [2].

1 Introduction

As is tradition for this course, our project for the semester was the challenge of last spring’s ACM SIGMOD Student Research Competition. The theme of the competition was the efficient and fast creation of a *K Nearest Neighbor* graph, where its vertices belong to a dataset of 100-dimensions vectors/datapoints of floats.

A brute force approach to this problem has a complexity of $O(n^2)$, since each datapoint needs to check all others so that it can *weed out* the K points nearest to it. An answer to this problem was given in 2011 by Dong et al. in their paper titled “Efficient K-Nearest Neighbor Graph Construction for Generic Similarity Measures” [4], where they present the *NN-Descent* algorithm. This algorithm is based at the very simple idea that “*a neighbor of a neighbor is also likely to be a neighbor*”. According to them, this algorithm is shown to have a complexity of $O(n^{1.14})$. This is a major step-up from the quadratic time, especially when it comes to big dataset.

Our library does not only implement the NN-Descent algorithm. It also implements other optimizations that

e.g. make the starting graph more like the one we want to end up with, lowering this way the number of iterations the algorithm must do to reach a satisfying result. These optimizations are discussed in sec. 2.

1.1 Parameters

The full algorithm can be tweaked by a number of parameters. The ones that have to do with the NN-Descent algorithm are: K for the number of nearest neighbors to search for, δ for the threshold to the early termination [4, sec. 2.6] and ρ for the sampling rate [4, sec. 2.5].

Apart from those, there are two other parameters that have to do with the creation of the starting graph when random projection trees are used (sec. 2.2): D for the upper bound to the number of vertices each leaf must have and T for the number of random projection trees to compute.

1.2 Project layout

- The library itself is stored in the `src/` directory. It is compiled to a `.a` file, so that it can be linked with any executable.

*Student ID: 1115202000215

†Student ID: 1115202000179

- The main app that results to the main executable is in the `app/` directory. This file contains the only `main()` on the project, and can be thought as the API between the user and the library itself.
- The directory `datasets/` contains a lot of the datasets of the SIGMOD competition in binary format. The subdirectory `computed/` contains some of the K-NN graphs that were computed using the brute force algorithm for comparison purposes. They are also in binary format. For which dataset and for which K they correspond to can be deduced from the respective filename.
- The `docs/` subdirectory contains the `html/` and `latex/` subdirectories that they themselves contain files respective to each format. It is documentation of the code, automatically generated by code comments using [Doxygen](#). The `lcov/` subdirectory contains files that are generated by [LCOV](#) and are used to monitor the code coverage provided by the tests (see below)¹,
- The `extern/` directory contains the source code for the [Google Test](#) suite. It is included as a git submodule.
- The `include/` directory contains header files for the library and the home-made data structures it uses.
- The `test/` directory contains unit tests for the library.
- The `helper.sh` file is a bash script that is used to automate some repetitive tasks, such as execution of all the tests, or documentation generation.
- The `experiments.sh` is a script that executes the main program in quiet mode, and outputs a `.csv` file that contains the results of different executions for different parameters (see sec. 3).

1.3 Data structures

1.3.1 Defunct Structures

1.3.1.1 ADTMap using HashTable In the first and second submission we implemented the ADTGraph using an adjacency ADTMap, a map where we stored a pair of vertices (edge) as a key that was related with its weight. For the implementation of this ADTMap we used a hash table. In the final submission, we removed the map data type as we find a simpler and more efficient

way to implement the graph data type.

1.3.1.2 ADTList using LinkedList In previous versions we also used ADTList, implemented using Linked List to get access to some of the data stored in the mentioned graph. This data type is not used anymore as well, as we replaced it by the more efficient ADTPriorityQueue in some cases and ADTVector in other where having random access to the elements could be helpful.

1.3.2 Used Structures

1.3.2.1 ADTGraph In the final version the ADTGraph is implemented using a logic similar to the adjacency matrix. More specifically, in the graph is stored an ADTVector that contains its vertices each one of which are represented by a class called GraphVertex. In this class, we keep a PriorityQueue which contains the vertex's direct neighbors and one that contains its reverse neighbors.

1.3.2.2 ADTPriorityQueue using Heap As we mentioned earlier, we use a PriorityQueue which is implemented by a heap data structure. In this data type we have added some functionality by implementing a remove function that removes from the queue the node that contains a value equivalent to a given one. The search in this case has time complexity $O(n)$, worst-case.

1.4 Library interface

- `Vector* readBinData(const char* fp, int dimensions):` reads data from a binary file and stores it in a vector.
- `void writeBinGraph(const char* fp, Graph* graph, int K):` writes an already computed graph in a binary file.
- `Graph* readBinGraph(const char* fp, int dimensions):` reads and returns a graph from a binary file.
- `float recall(Graph* bfGraph, Graph* nnGraph, int N, int K):` returns the recall of the graph computed by NN-Descent, compared to the brute force graph.
- `float euclideanDistance(Pointer a, Pointer b):` metric function that computes the euclidean distance between 2 points.

¹At the moment, the code coverage of the `cppdescent` library itself is pretty low, since it went very recently under a large refactoring and overhaul.

- `float manhattanDistance(Pointer a, Pointer b)`: metric function that computes the manhattan distance between 2 points.
- `Graph* KNNBruteForceGraph(Vector* data, int K, CompareFunc compare)`: function that computes the K-NN graph using brute force.
- `Graph* NNDescent_KNNGraph(Vector* data, int K, int D, int Trees, float delta, float rho, DistanceFunc distance)`: function that computes the K-NN graph for the given dataset using the NN-Descent algorithm.
- `PQueue* NNDescent_Query(Graph* graph, int K, CompareFunc compare, Vector* query)`: function that computes the K Nearest Neighbors of the query point in the graph.
- `void RPTree(Graph* graph, Vector* vec, int K, int D, int dimensions)`: creates a graph using random projection trees.

$$d_{\mathbf{x},\mathbf{y}} = \sqrt{(x_1 - y_1)^2 + \dots + (x_N - y_N)^2} \quad (1)$$

The first thing we observe is that for the needs of the algorithm, the square root is unnecessary. The resulting expression can be then rewritten as vector operations, like so:

$$ds_{\mathbf{x},\mathbf{y}} = \mathbf{x}^2 + \mathbf{y}^2 - 2\mathbf{xy},$$

where $\mathbf{x}^2$ and $\mathbf{y}^2$ are the square euclidean norms of the respective vectors. This gives us the option to compute these values only once for each vector and save them, saving precious computing time.

For the computation of the euclidean norms, the dot product of $\mathbf{xy}$, but also for the representation of the vectors themselves the GNU Scientific Library [7] is used.

2 Optimizations

Since this project was developed in three stages, each stage called for new optimizations upon the code of the previous one. The second assignment/submission was focused on the optimizations proposed by Dong et al. Namely *local join*, *incremental search*, *sampling* and *early termination*. Each of those is presented extensively on the NN-Descent paper, so there is no need to do the same here.

On the other hand, the third assignment called for some more interesting optimizations: By utilizing linear algebra, it is possible to simplify the computation of a distance between two vectors by a lot. Another optimization is a different approach on the creation of the starting graph, from which the NN-Descent algorithm starts running iteratively. Lastly, a very simple and obvious optimization is making use of the powerful computing systems of today and introducing parallelization/multi-threading to the algorithm.

2.1 Precomputed vector norms

Finding the distance between two points is a big chunk of this algorithm's computing time. Especially when we are talking about the *euclidean* distance for 100-dimensional points (eq. 1).

2.2 Random Projection Trees

Before the algorithm can begin improving a given graph, first a graph must be given. The default strategy is to draw K random edges for each vertex in the newly read graph, thus getting a totally random head-start.

A more sophisticated technique is the usage of random projection trees [3] for the initialization of said graph. The idea is simple: For the given dataset, we take a line (actually hyperplane for dimensions > 2) and split it in two halves. Then, we do the same thing recursively until we have at most D datapoints on each split — these final areas are the tree's *leaves*.

3 Experiments

3.1 Experimental Setup

3.1.1 Performance Measures

3.1.2 Default Parameters

3.1.3 System Environment

3.2 Experimental Results

3.2.1 Performance

3.2.1.1 Execution Speed

3.2.1.2 Memory Consumption

3.2.2 Performance as Data Scales

3.2.3 Tweaking the Parameters

4 Conclusion

5 Acknowledgements

3.2.1.3 Single vs. Multi-Threading

References

- [1] J. Brugger, *Brj0/nndescent*, Dec. 11, 2023. [Online]. Available: <https://github.com/brj0/nndescent>.
- [2] K. Chousos, *Kchousos/CPDDescent*, Jan. 18, 2024. [Online]. Available: <https://github.com/kchousos/CPDDescent>.
- [3] S. Dasgupta and Y. Freund, “Random projection trees and low dimensional manifolds,” in *Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing*, Victoria British Columbia Canada: ACM, May 17, 2008, pp. 537–546, ISBN: 978-1-60558-047-0. DOI: 10.1145/1374376.1374452. [Online]. Available: <https://dl.acm.org/doi/10.1145/1374376.1374452>.
- [4] W. Dong, C. Moses, and K. Li, “Efficient k-nearest neighbor graph construction for generic similarity measures,” in *Proceedings of the 20th International Conference on World Wide Web*, ser. WWW ’11, New York, NY, USA: Association for Computing Machinery, Mar. 28, 2011, pp. 577–586, ISBN: 978-1-4503-0632-4. DOI: 10.1145/1963405.1963487. [Online]. Available: <https://dl.acm.org/doi/10.1145/1963405.1963487>.
- [5] Enthought, director, *PyNNDescent Fast Approximate Nearest Neighbor Search with Numba* | *SciPy 2021*, Jul. 23, 2021. [Online]. Available: https://www.youtube.com/watch?v=xPadY4_kt3o.
- [6] *Google/googletest*, Google, Jan. 18, 2024. [Online]. Available: <https://github.com/google/googletest>.
- [7] “GSL - GNU Scientific Library - GNU Project - Free Software Foundation.” (), [Online]. Available: <https://www.gnu.org/software/gsl/>.
- [8] “How PyNNDescent works — pynndescent 0.5.0 documentation.” (), [Online]. Available: https://pynndescent.readthedocs.io/en/latest/how_pynndescent_works.html.
- [9] Γ. Ιωαννίδης. “Ανάπτυξη Λογισμικού για Πληροφοριακά Συστήματα.” (), [Online]. Available: <https://eclass.uoa.gr/courses/DI485/>.
- [10] D. Kluser, J. Bokstaller, S. Rutz, and T. Buner. “Fast Single-Core K-Nearest Neighbor Graph Computation.” arXiv: 2112.06630 [cs, math]. (Dec. 13, 2021), [Online]. Available: <http://arxiv.org/abs/2112.06630>, preprint.
- [11] OpenMP Architecture Review Board, *OpenMP application program interface version 5.2*, Nov. 2021. [Online]. Available: <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf>.
- [12] Stitch Fix Multithreaded, director, *Algo Hour - Nearest Neighbor Descent (and friends)* | *Dr. Leland McInnes*, May 22, 2020. [Online]. Available: https://www.youtube.com/watch?v=0vT2NY_FV_g.